TRILLIUM
INFORMATION SECURITY SYSTEMS

Table of Contents

Report on Demonstrating

Task Overview

Create a C# application that download file from remote server on the basis of given file ID. Create a python script that download file from 1-200 ID.

Solution:

/file_server

create_sample_files.py

What the Script Does

- **Directory Creation**: The script first checks if the files directory exists. If not, it creates it using os.makedirs(files_directory, exist_ok=True). The exist_ok=True parameter ensures that no error is raised if the directory already exists.

- **File Generation**: The script generates 200 files named 1.txt to 200.txt inside the files directory. Each file contains sample content, which can be customized as needed.
- **Confirmation**: After running the script, it confirms the creation of the files.
- **import os**: The os module provides functions for interacting with the operating system, such as creating directories and handling file paths.
- **os.makedirs(files_directory, exist_ok=True)**: This creates the directory 'files' if it doesn't already exist. The exist_ok=True parameter ensures that the script doesn't throw an error if the directory is already present.
- **File Generation Loop (for i in range(1, 201))**: This loop generates 200 files named 1.txt, 2.txt, ..., 200.txt. For each file, the script writes a line of text, This is the content of file {i}.
- **os.path.join(files_directory, f'{i}.txt')**: This joins the directory and filename to create a valid file path.

```
import os

def create_files_directory():
    # Define the directory name
    files_directory = 'files'

    # Create the directory if it doesn't exist
    os.makedirs(files_directory, exist_ok=True)

    # Generate files from 1.txt to 200.txt
    for i in range(1, 201):
        file_path = os.path.join(files_directory, f'{i}.txt')

        # Write sample content into each file
        with open(file_path, 'w') as file:
            file.write(f'This is the content of file {i}.\n')

    print(f'{200} sample files have been created in the "{files_directory}" directory.')

if __name__ == '__main__':
    create_files_directory()
```

```
PS C:\Users\ammok\Desktop\Ascend Internship Tasks\Task#3-C# application\file_server> python
create_sample_files.py
```

server.py File:

Functionality:

- **File Serving:** The server.py file creates a Flask server that serves files from the files directory. The server is set up to listen for requests at http://localhost:5000/files/<file_id>, where <file_id> is the ID of the file to be downloaded.

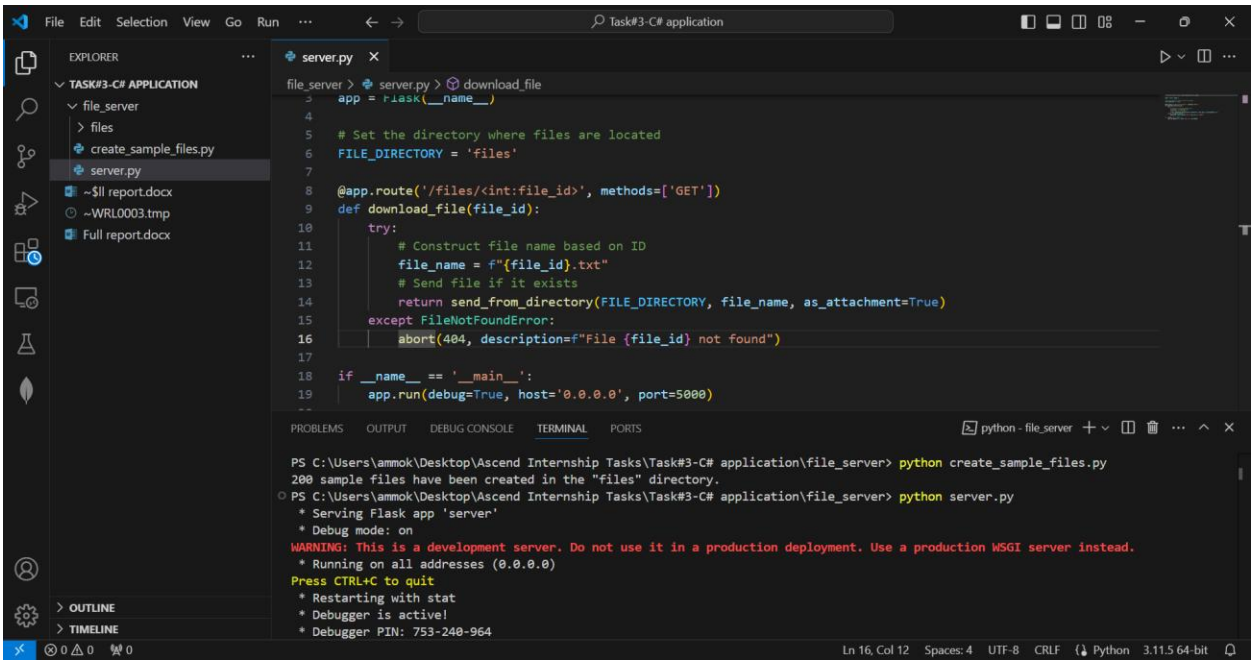
- **Error Handling:** If the requested file is not found, a 404 error is returned.

- **Flask Setup (Flask(__name__))**: This line initializes a Flask application. Flask is a web framework that allows Python applications to respond to web requests.
- **@app.route('/files/<path:file_name>', methods=['GET'])**: This defines the URL route for downloading files. The <path:file_name> indicates that the URL will take the file name as a parameter.
- **send_from_directory(FILE_DIRECTORY, file_name, as_attachment=True)**: This function sends a file from the specified directory. The as_attachment=True flag ensures that the file is downloaded rather than displayed in the browser.
- **Error Handling (abort(404))**: If the file isn't found, the server responds with a 404 error.

```
from flask import Flask, send_from_directory, abort

app = Flask(__name__)

# Set the directory where files are located
FILE_DIRECTORY = 'files'

@app.route('/files/<path:file_name>', methods=['GET'])
def download_file(file_name):
    try:
        # Send file if it exists
        return send_from_directory(FILE_DIRECTORY, file_name, as_attachment=True)
    except FileNotFoundError:
        abort(404, description=f"File {file_name} not found")

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

```
python server.py
```

- The server will start at http://localhost:5000 .



/file_server/files

this is the folder that is automatically created by the script .py above saved.

Client Side

download_files.py

Functionality:

- **Single File Download:** Downloads a single file based on the file ID provided.
- **Batch Download:** Downloads files in a range of IDs from 1 to 200.
- **requests.get(file_url)**: This sends a GET request to the server to download the file. If successful, the content is written to a local file.
- **Saving Files**: The files are saved in a downloads directory, with each file being prefixed by downloaded_.
- **User Interaction**: The script prompts the user to choose an option (download all files, download multiple, or download one). Based on the input, the appropriate files are downloaded.

```
import requests
import os

def download_file(file_name, save_directory):
    server_url = "http://localhost:5000/files"
    file_url = f"{server_url}/{file_name}"

    try:
        response = requests.get(file_url)
        response.raise_for_status()
```

```
        save_path = os.path.join(save_directory, f"downloaded_{file_name}")
        with open(save_path, 'wb') as file:
            file.write(response.content)

        print(f"File {file_name} downloaded successfully.")
    except requests.exceptions.RequestException as e:
        print(f"Failed to download file {file_name}: {e}")

def download_files(file_names, save_directory):
    for file_name in file_names:
        download_file(file_name, save_directory)

def get_user_choice():
    print("Choose an option:")
    print("1. Download all files from 1.txt to 200.txt")
    print("2. Download multiple files")
    print("3. Download a single file")
    return input("Enter your choice (1, 2, or 3): ")

def handle_download_choice(choice):
    save_directory = 'downloads'
    os.makedirs(save_directory, exist_ok=True)

    if choice == '1':
        file_names = [f"{i}.txt" for i in range(1, 201)]
        download_files(file_names, save_directory)
    elif choice == '2':
        file_names_input = input("Enter the file names separated by commas (e.g., 1.txt, 3.txt, 7.txt): ")
        file_names = [name.strip() for name in file_names_input.split(',')]
        download_files(file_names, save_directory)
    elif choice == '3':
        file_name = input("Enter the file name to download (e.g., 1.txt): ").strip()
        download_file(file_name, save_directory)
    else:
        print("Invalid choice. Please select 1, 2, or 3.")

if __name__ == "__main__":
    user_choice = get_user_choice()
    handle_download_choice(user_choice)
```

```
python download_files.py
```

Program.cs

Functionality:

- **File Downloading:** The C# application prompts the user to enter a file ID, constructs the URL, and downloads the file from the server.

Libraries Used

- using System;:

  This namespace provides fundamental classes and base types (like Console, String, etc.). In this case, it is used for handling input/output operations with Console.WriteLine() and Console.ReadLine().

- using System.IO;:

  This namespace is used for file handling, such as reading from or writing to files. In this code, it's used with File.WriteAllBytesAsync() to save the downloaded file to the disk.

- using System.Net.Http;:

  This namespace provides classes for sending HTTP requests and receiving HTTP responses. In this case, it's used for downloading files from a remote server using the HttpClient class.

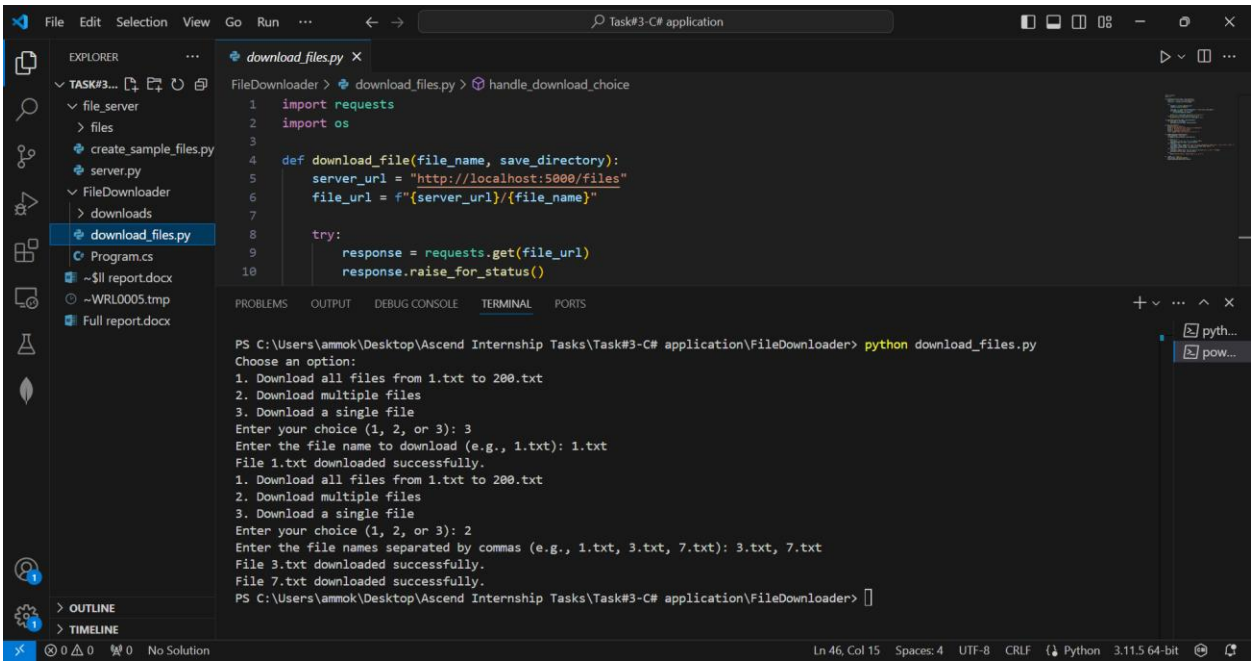
- using System.Threading.Tasks;

  This namespace provides support for asynchronous programming. The async and await keywords used in the program are part of this namespace, which allow the program to perform tasks without blocking the main thread.

```
using System;
using System.IO;
using System.Net.Http;
using System.Threading.Tasks;

class FileDownloader
{
   private static readonly HttpClient client = new HttpClient();

   static async Task Main(string[] args)
   {
      Console.WriteLine("Enter the File ID to download:");
      string fileId = Console.ReadLine();

      string serverUrl = "http://localhost:5000/files"; // Use your server URL here
      string fileUrl = $"{serverUrl}/{fileId}";

      try
      {
         await DownloadFileAsync(fileUrl, fileId);
         Console.WriteLine("File downloaded successfully.");
      }
      catch (Exception ex)
      {
         Console.WriteLine($"Error: {ex.Message}");
      }
   }
```

```
    private static async Task DownloadFileAsync(string url, string fileId)
    {
        HttpResponseMessage response = await client.GetAsync(url);
        response.EnsureSuccessStatusCode();

        byte[] fileBytes = await response.Content.ReadAsByteArrayAsync();
        string fileName = $"downloaded_file_{fileId}.dat"; // You can adjust the file extension

        await File.WriteAllBytesAsync(fileName, fileBytes);
    }
}
```

Output



Summary

- **Server Setup:** A Flask server is used to host files, accessible at http://localhost:5000/files/<file_id>.
- **Python Script:** Automates the download of files with IDs from 1 to 200.
- **C# Application:** Allows users to download a specific file based on the provided file ID.

Application in Offensive Cybersecurity

1. **File Retrieval Techniques:** Understanding how to set up and interact with servers for file retrieval can be essential in offensive cybersecurity for exploiting file download vulnerabilities or

performing unauthorized data exfiltration (Data exfiltration—also known as data extrusion or data exportation—is data theft: the intentional, unauthorized, covert transfer of data from a computer or other device. Data exfiltration can be conducted manually, or automated using malware).

2. **Vulnerability Testing:** The scripts demonstrate how to set up a server and client for file downloads, which can be used to test file download and access vulnerabilities in applications. For example, a similar setup could be used to test for path traversal vulnerabilities, where an attacker might attempt to download files outside of the intended directory.
3. **Proof of Concept:** This setup can be used as a proof of concept for demonstrating file download functionality and potential security weaknesses, such as improper file handling or inadequate error handling.
4. **Automation in Penetration Testing:** Automating file download processes can streamline various penetration testing tasks, such as bulk file extraction or testing file retrieval mechanisms across different IDs.

Overall, this approach aligns with the task requirements, showing both the server setup for handling file requests and the client applications for downloading files, which are crucial for understanding file-based vulnerabilities and exploitation in the cybersecurity domain.